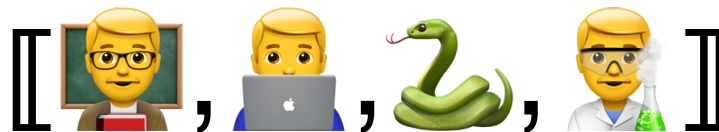


Lecture Notes for **Neural Networks I**

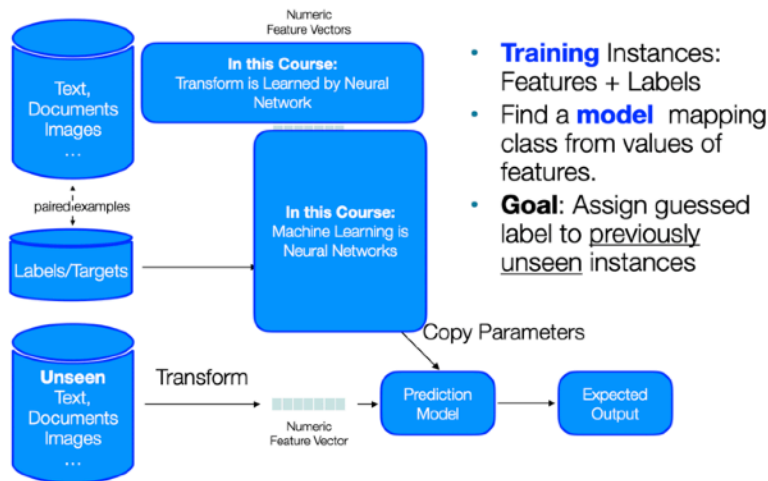


Professor Eric Larson
Table Data with Numpy and Pandas

Class Logistics and Agenda

- Canvas. Videos. Attendance. Participation.
- Agenda:
 - Data Encodings
 - Demo: Table Data, Numpy, Pandas
 - Data Quality and Imputation

Last Time



Most Install Instructions (Windows, Linux, and Intel-base Macs)

For operating systems that do NOT use Apple Silicon, you can use the following commands:

```
conda create --name mlenv3_12 python=3.12
```

```
conda activate mlenv3_12
```

```
pip install jupyter numpy scipy pandas matplotlib scikit-learn plotly missingno
```

```
pip install protobuf tensorflow-datasets importlib-resources
```

```
pip install torch torchvision transformers
```

Apple Silicon MacOS Install Instructions

Note: Apple M-series Macs should use a different install procedure as follows:

```
conda create --name mlenv3_12 python=3.12
```

```
conda activate mlenv3_12
```

```
python -m pip install tensorflow-macos
```

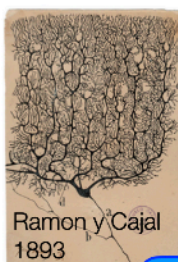
```
python -m pip install tensorflow-metal
```

```
pip install jupyter pandas matplotlib scikit-learn plotly missingno
```

```
pip install "numpy<2.0" "scipy<=1.13.1"
```

```
pip install protobuf tensorflow-datasets importlib-resources
```

```
pip install torch torchvision transformers
```



Biologically Inspired

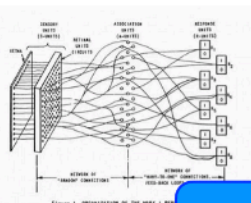


Turns into Optimization of Matrices

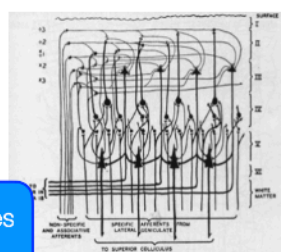
...



4. Frank Rosenblatt with his Mark I perceptron (left), and a graphical representation of it (right).
<https://data-science-blog.com/blog/2023/07/16/a-brief-history-of-neural-nets-everything-you-need-to-know-before-learning-lets/>



Fundamentally guides Connectivity



Class Overview, by topic

Table Data
Visualization

Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Linear and
Logistic
Regression

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Neural Networks
and Back Prop.

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Wide and Deep
Networks

Convolutional
Networks

Sequential
Networks

Keras, Tensorflow, Torch
Intuition, Detailed implement.

Large Language
Models (if time)

Hugging Face
Intuition only

Types of Data and Categorization



Table Data

- **Table Data:** Collection of data **instances** and their **features**

- **Python:** Pandas Dataframe
- **R:** Data.frame
- **Matlab:** Table Class
- **C++:** Make your own,
`std::vector<Record>`

Self Test: Table data storage is not memory efficient.

- a) True
- b) False
- c) It depends on the backend
- d) It depends on the programming language of the interface

Attributes, columns,
variables, fields,
characteristics, **Features**

Target,
Class,
Label

Objects,
records,
rows,
points,
samples,
cases,
entities,
instances

	<i>Pregnant</i>	<i>BMI</i>	<i>Age</i>	<i>Diabetes</i>
1	Y	33.6	41-50	positive
2	N	26.6	31-40	negative
3	Y	23.3	31-40	positive
4	N	28.1	21-30	negative
5	N	43.1	31-40	positive
6	Y	25.6	21-30	negative
7	Y	31.0	21-30	positive
8	Y	35.3	21-30	negative
9	N	30.5	51-60	positive
10	Y	37.6	51-60	positive

Feature Vector Representation

$f_{mono}(\cdot)$
monotonic function

Discrete

Categorical or Nominal

Variable could be one value in a set of categories. No ordering.

Example: Employee ID

Allowed Transforms:
permuting values

**boolean, one hot encoding,
or hash (as integer)**

1	0	0	→0
0	1	0	→1
0	0	1	→2

Ordinal

Variable could be one value in a set of categories. Ordering matters.

Example: Star Ratings, 1-5

Allowed Transforms:

$$V_{new} = f_{mono}(V_{old}) + b$$

integer (or boolean)

Continuous

Interval or Numeric

Value is continuous numeric value. Could be in specified range.

Example: BMI, Temperature, etc.

Allowed Transforms:

$$V_{new} = f_{mono}(V_{old}) + b$$

float

Ratio or Numeric

Value is continuous numeric value. Zero is meaningful. Often not treated differently than interval.

Example: Length, Elevation

Allowed Transforms:

$$V_{new} = f_{mono}(V_{old})$$

float

Data Tables as Variable Representations

Table

<i>TID</i>	<i>Pregnant</i>	<i>BMI</i>	<i>Age</i>	<i>Eye Color</i>	<i>Diabetes</i>
1	Y	33.6	41-50	brown	positive
2	N	26.6	31-40	hazel	negative
3	Y	23.3	31-40	blue	positive
4	N	28.1	21-30	brown	inconclusive
5	N	43.1	31-40	blue	positive
6	Y	25.6	21-30	hazel	negative

Internal Rep.

<i>TID</i>
1
2
3
4
5
6

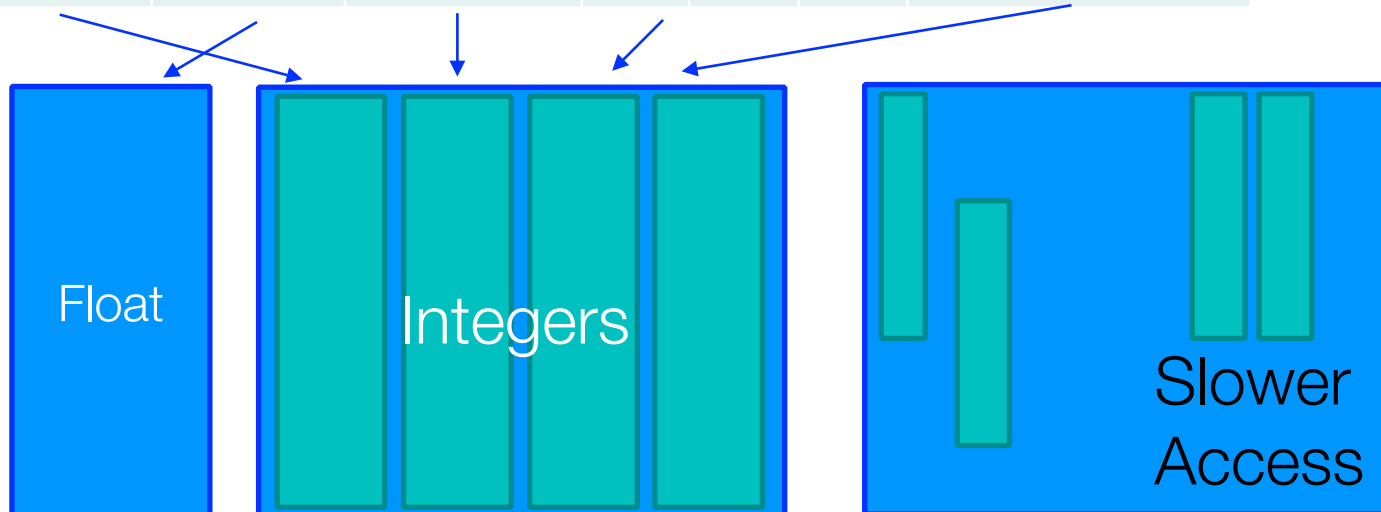
Data Tables as Variable Representations

Internal Rep.

<i>TID</i>	<i>Binary</i>	<i>Float</i>	<i>Ordinal</i>	<i>One Hot</i> brown hazel blue			<i>Diabetes</i>
1	1	33.6	2	1	0	0	1
2	0	26.6	1	0	1	0	0
3	1	23.3	1	0	0	1	1
4	0	28.1	0	1	0	0	2
5	0	43.1	1	0	0	1	1
6	1	25.6	0	0	1	0	0

Memory

Fast access via contiguous memory blocks, ideally



“Finish” Jupyter Notebooks

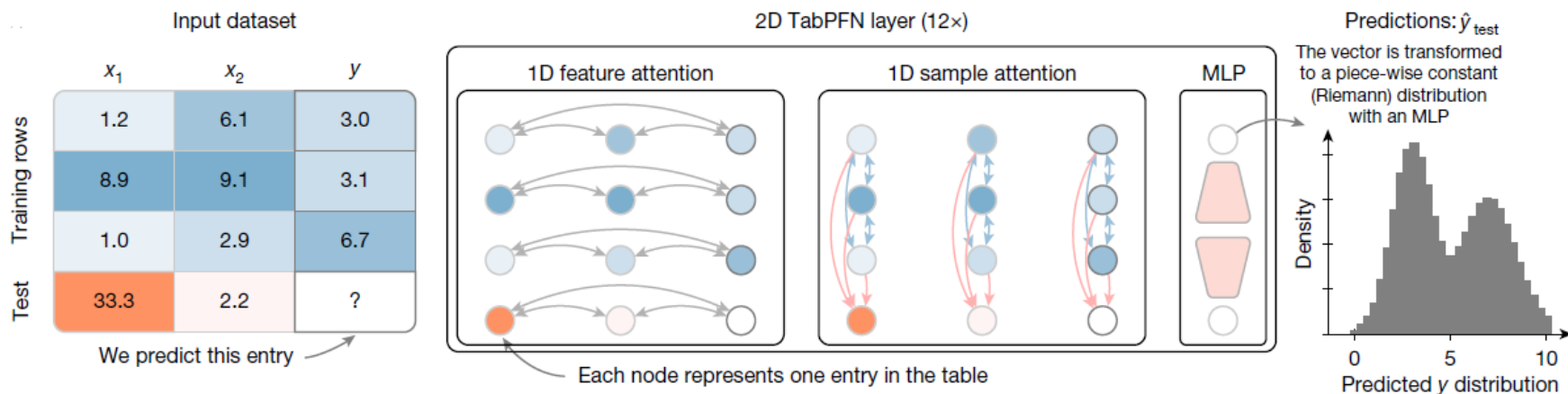


01. Numpy and Pandas Intro.ipynb

Foundational Models

- For many types of data, foundational models:
 - Pre-trained on large datasets
 - Used as starting points for adapting to new data
 - Efficient and straightforward for neural networks
 - **Text:** BERT, GPT, Llama
 - **Images:** ResNet, ConvNext, Swin, ViT, ... YOLO

TabPFN: Foundational Model for Table Data ...



Data Quality



Dear Claude Code User,

Thank you for opting in to sharing your data, we appreciate you helping us make Claude better for everyone.

Upon reviewing your code quality, we have decided to opt you out of this programme.

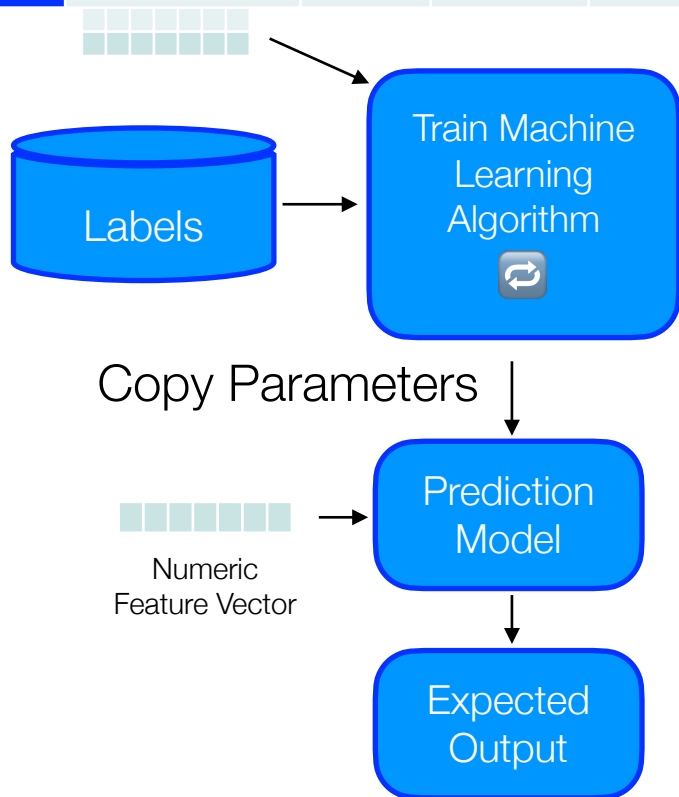
Warmly,

The Anthropic Team

Data Quality Problems

TID	Hair Color	Hgt.	Age	Arrested
1	Brown	5'2"	23	no
2	Hazel	1.5m	12	
3	NaN	5	999	no
4	Brown	5'2"	23	no

- Missing
 - Easy to find, NaNs
- Duplicated
 - Easy to find, hard to verify
- Noise or Outlier
 - Hard to define / catch



Information is not collected
(e.g., people decline to give their age and weight)

Features **not applicable**
(e.g., annual income for children)

UCI ML Repository: 90% of repositories have missing data

Handling Issues with Data Quality

- **Eliminate** Instance or Feature
- **Ignore** the Missing Value During Analysis Replace with all possible values (talk about later)

- **Impute** Missing Values

How?

Stats?
mean
median
mode

Imputation

- When is it probably fine to impute missing data:
 - (A) When there is not much missing data
 - (B) When the missing feature is mostly predictable from other features
 - (C) When there is not much missing data for some subgroup of the data
 - (D) When it is the class you want to predict

Split-Impute-Combine



<i>TID</i>	<i>Pregnant</i>	<i>BMI</i>	<i>Age</i>	<i>Diabetes</i>
1	Y	33.6	31-40	positive
2	N	26.6	31-40	negative
3	Y	23.3	?	positive
4	N	28.1	21-30	negative
5	N	43.1	31-40	positive
6	Y	25.6	21-30	negative
7	Y	31.0	21-30	positive
8	Y	35.3	?	negative
9	N	30.5	51-60	positive
10	Y	37.6	21-30	positive

Try to group similar rows together first, then impute within the group

split: pregnant
split: BMI > 32

<i>TID</i>	<i>Pregnant</i>	<i>BMI</i>	<i>Age</i>	<i>Diabetes</i>
1	Y	>32	31-40	positive
8	Y	>32	?	negative
10	Y	>32	21-30	positive

Mode: none, can't impute

<i>TID</i>	<i>Pregnant</i>	<i>BMI</i>	<i>Age</i>	<i>Diabetes</i>
3	Y	<32	?	positive
6	Y	<32	21-30	negative
7	Y	<32	21-30	positive

Mode: 21-30